

Compiling with Counterexamples:

correctness, copying, cost and comprehension for partial compilers

Valeria de Paiva

August 21, 2026

Abstract

A partial compiler — one that reduces a source program, delegates, and assembles the target from whatever comes back — is a Dialectica morphism with its relation discarded. Restoring the relation, with values in a lineale rather than in the booleans, gives compositional correctness at no cost in the data: being correct is a condition a partial compiler either meets or fails, never extra data it has to carry, and a stack of passes is verified by pushing the specification of the target back through the passes one at a time, rather than by proving anything about the composite. Three further consequences follow by unfolding definitions. Whether a partial compiler may duplicate its source is a question about the target type alone, which accounts for a failure of associativity they report when compile-time effects are moved inside. Whether it can carry a cost model is settled by which equations that type satisfies. And restricting to sources for which the specification holds makes the collection of verified partial compilers an object rather than a class, right adjoint to the obvious product.

1 Partial compilers

A compiler need not do the whole job. It may perform one pass — lowering a loop, partitioning a computation across machines, choosing a representation — and leave the rest to somebody else. To describe that, Budiú, Galenson and Plotkin [1] introduce the *partial compiler*: given a source program it produces a reduced source, to be handed on, together with a way of assembling the final target from whatever target comes back:

$$f: \text{source} \rightarrow \text{source}', \quad F: \text{source} \times \text{target}' \rightarrow \text{target}. \quad (1)$$

Both halves are needed, and the first argument of F is the point of the definition: assembling the final target generally requires the original source, not merely the target returned, since the material the pass set aside has to be put back.

Partial compilers plug into one another. Handing the reduced source of one to another and assembling in the reverse order gives

$$(f, F); (g, G) = (g \circ f, (u, z) \mapsto F(u, G(f(u), z))), \quad (2)$$

a partial compiler again, and the operation is associative with an evident identity. So there is a category, whose objects are pairs $(\text{source}, \text{target})$ — or, in the vocabulary those authors use, pairs of a *problem* and a type of *solutions* to it.

Composition is not the only operation they define. There is a tensor, running two partial compilers side by side; a star, applying one to each element of a list; conditionals and case analysis; and a *functor* operation $\text{PCFunc}(f, g)$, assembled from a map f on sources and a map g on targets, whose generation function applies g and ignores the source. Only the tensor and this last are needed below.

That vocabulary is worth pausing on. Reducing a problem to another problem, and assembling a solution of the first from a solution of the second, is Kolmogorov’s reading of intuitionistic logic as a calculus of problems, and the n -ary version is Milner’s tactics: a goal is reduced to subgoals and a proof is assembled from proofs of those. Partial compilers are the same shape at a different scale, and what follows applies verbatim to all three readings. The Kolmogorov reading is more than suggestive: Kolmogorov’s problems, Veloso’s abstract problems and Blass’s questions and answers are related to one another precisely by Dialectica constructions [10], so the same construction that organises the notions of problem is the one applied to compilers here.

Nothing in the definition says the compiler is correct. That is the gap this note fills.

2 The Dialectica construction

Fix a cartesian closed $\mathcal{C}$ with finite coproducts. Gödel’s Dialectica interpretation assigns to a formula A a formula $\exists u \forall x \alpha(u, x)$ with α quantifier-free, where u ranges over *witnesses* — proposed evidence for A — and x over *counterexamples*, challenges to a proposed witness. Proving an implication yields, beyond a map on witnesses, a way of turning a challenge to the conclusion into a challenge to the premiss.

The categorical construction takes that shape as its objects. Let L be a *lineale*: an object of truth values carrying an order $\sqsubseteq$, a symmetric monoidal $\otimes$ with unit e that is monotone in each argument, and a right adjoint $\multimap$ to it. Read $a \sqsubseteq b$ as a entailing b . The two-element boolean algebra, with $\otimes = \wedge$, is the classical case. An object of $\mathcal{D}_L(\mathcal{C})$ is a triple (U, X, α) with $\alpha: U \times X \rightarrow L$; a morphism to (V, Y, β) is

$$f: U \rightarrow V, \quad F: U \times Y \rightarrow X, \quad \text{with} \quad \alpha(u, F(u, y)) \sqsubseteq \beta(f(u), y). \quad (3)$$

Witnesses go forwards, counterexamples backwards, and the condition is Gödel’s clause for implication: if the challenge manufactured from y defeats u , then y defeats $f(u)$. We write $\ell = (f, F)$ for such a pair, and call it a morphism or, when the compiler reading is meant, a pass.

The construction is the original ‘dialectica’ category of de Paiva [2, 3]. It is symmetric monoidal closed, with tensor computed coordinatewise,

$$(U, X, \alpha) \otimes (V, Y, \beta) = (U \times V, X \times Y, ((u, v), (x, y)) \mapsto \alpha(u, x) \otimes \beta(v, y)), \quad (4)$$

and unit $I = (\mathbf{1}, \mathbf{1}, e)$. The coordinatewise formula is worth writing out, because a reader arriving from Chu spaces or from optics will expect the function spaces $X^V \times Y^U$ in the second coordinate. Those belong to the variant $\mathcal{G}_L(\mathcal{C})$ described below, where the backward map does not see the witness; once F takes U as an argument the exponentials are already absorbed, as currying $\text{Hom}(A \otimes B, C) \cong \text{Hom}(A, B \multimap C)$ shows. The distinction is not cosmetic here: §5 turns on duplication being a question about X alone, which is what the

coordinatewise tensor says and the crossed one denies. The cartesian product, which §7 needs, sums in the second coordinate instead:

$$(U, X, \alpha) \times (V, Y, \beta) = (U \times V, X + Y, ((u, v), \text{inl } x) \mapsto \alpha(u, x), ((u, v), \text{inr } y) \mapsto \beta(v, y)), \quad (5)$$

a challenge to a pair being a challenge to one component or to the other.

Two further pieces of the construction are worth naming now, though neither is used before §5. The first is the variant $\mathcal{G}_L(\mathcal{C})$: the same objects, but a morphism $(U, X, \alpha) \rightarrow (V, Y, \beta)$ is

$$f: U \rightarrow V, \quad F: Y \rightarrow X, \quad \text{with} \quad \alpha(u, F(y)) \sqsubseteq \beta(f(u), y), \quad (6)$$

the backward map now independent of the witness. It is the fragment in which nothing about the source flows backwards, and §3 identifies one of their operations on partial compilers with it. The second is the modality: $\mathcal{D}_L(\mathcal{C})$ carries a comonad $!$ which replaces the counter object X by the free commutative monoid on it. Proposition 5 says which objects already have the structure that $!$ supplies freely.

3 Partial compilers are Dialectica morphisms

Displays (1) and (3) are the same data, and switching the relation off makes the condition of (3) vacuous. But the agreement is not only at the level of the data: the categorical structure matches item by item.

Proposition 1. *For $L = \mathbf{1}$ the condition in (3) holds always, and $\mathcal{D}_1(\mathcal{C})$ is isomorphic to the category of partial compilers of Budiu, Galenson and Plotkin by the isomorphism that is the identity on data: an object $(U, X, !)$ is a pair (source, target), a morphism (f, F) is a reduction together with a generation function, and (2) is Dialectica composition.*

The identity (id_U, π_2) is their identity partial compiler, their tensor is (4), and their functor operation is the inclusion of $\mathcal{G}_1(\mathcal{C})$, in which the backward map ignores the source. Those authors say as much from their side: for the identity monad their category of tactics is equivalent to the Dialectica category. The identification was set out in a talk on their preprint in 2011 [7].

So a partial compiler is a Dialectica morphism whose relation has been thrown away. What that relation gives back once it is restored is the question this note answers, and the four sections that follow are the four answers.

Remark 2 (the second vocabulary). The same data is a *bimorphic lens* [8, 9]: $\text{get}: U \rightarrow V$ and $\text{put}: U \times Y \rightarrow X$, the abstraction of a field of a record, with put needing the old state because everything outside the field must be carried along. Monomorphic lenses, $U = X = S$ and $V = Y = A$, are the familiar case. The compiler reading is the one used here, because it is the one with an external correctness criterion attached: a target is right or wrong for a source by comparison with a semantics, whereas the lens laws are coherence between the two directions rather than agreement with a specification. That Dialectica categories and lenses are two views of one construction is by now well understood; the observation that lenses are Dialectica morphisms appears in Hedges. What seems not to have been said is that partial compilers belong on that list: Budiu, Galenson and Plotkin arrive at Dialectica through Blass’s problems and Milner’s tactics, and do not mention lenses.

4 Correctness

Read $\alpha(S, T)$ as saying that T is a *wrong* implementation of S : a refutation relation. Nothing in the construction settles this, and the literature reads the Dialectica relation both ways — affirmatively, with $\alpha(u, x)$ saying that u withstands the challenge x , and asymmetrically, with assertion on the witness coordinate and refutation on the counterexample one. For compilers the refutation reading is the one that works, and (3) is the first reason. Under it the condition reads:

if the target assembled for S is wrong, then the target returned for the reduced source $f(S)$ was already wrong.

That is compositional correctness for a partial compiler, on the nose and with no reformulation: the pass introduces no errors of its own. Attempting the affirmative reading, with $\alpha(S, T)$ saying that T is correct for S , points the condition the wrong way; the refutation reading is what makes the variance come out right. Display (7) below is the second reason.

A compiler that delegates nothing is the degenerate case, and it is worth spelling out because it is where the word “correct” acquires a definition. Suppose L has a least element $\perp$ and write $\mathbf{0} = (\mathbf{1}, \mathbf{1}, \perp)$. A morphism $(U, X, \alpha) \rightarrow \mathbf{0}$ consists of the unique map $U \rightarrow \mathbf{1}$ together with a backward map $U \times \mathbf{1} \rightarrow X$ — that is, nothing but a compiler $C: \text{source} \rightarrow \text{target}$ — and its condition reads

$$\alpha(u, C(u)) \sqsubseteq \perp, \quad (7)$$

that the target C assigns to a source is never a counterexample to it. So a correct compiler is not a compiler with a hypothesis attached: it is a morphism, and an arbitrary compiler is merely a map of $\mathcal{C}$ that fails to be one. Replacing $\perp$ by an arbitrary $\delta \in L$ gives the compilers whose refutations are bounded by δ , which is what §6 needs when L measures cost rather than truth. Note that the tensor unit $I = (\mathbf{1}, \mathbf{1}, e)$ is the wrong object here: when e is the top of L the condition $\sqsubseteq e$ is satisfied by everything.

Suppose now that a pass delegates to a child, and that we know what specification the child’s output must meet. What must the pass’s own source end satisfy? There is a best answer, and it is computed rather than guessed at.

Let $\ell = (f, F)$ be a pass from (U, X) to (V, Y) , and let L be complete. The child’s specification is a relation β on $V \times Y$; we are looking for a relation on $U \times X$, so we must say, for a source u and a candidate counterexample x to it, how bad x is allowed to be.

Now ℓ only ever produces the counterexample x by assembling it from something the child returned: the counterexamples y that ℓ turns into x at u are exactly those with $F(u, y) = x$. So x is only as much of a threat to u as the worst of those y is to $f(u)$, and we set

$$\ell^\# \beta(u, x) = \bigwedge \{ \beta(f(u), y) : F(u, y) = x \}, \quad (8)$$

taking the meet over that set, and the top of L when the set is empty — nothing the child returns is assembled into x , so x is no threat at all. In the boolean case (8) says: x refutes u only if every y that ℓ would turn into x already refutes $f(u)$. Transporting in the other direction instead,

$$\ell_! \alpha(v, y) = \bigvee \{ \alpha(u, F(u, y)) : f(u) = v \} \quad (9)$$

collects what a specification on the source end forces at the child’s.

Proposition 3. $\ell_! \dashv \ell^\#$, and ℓ is a morphism $(U, X, \alpha) \rightarrow (V, Y, \beta)$ if and only if $\alpha \sqsubseteq \ell^\# \beta$, equivalently $\ell_! \alpha \sqsubseteq \beta$. Moreover $(\ell' \ell)^\# = \ell^\# \ell'^\#$.¹

So $\ell^\# \beta$ is the weakest specification a source end can carry while leaving ℓ correct with respect to β — a weakest precondition, with (3) as the Hoare triple for a program that runs in two directions. The equation $(\ell' \ell)^\# = \ell^\# \ell'^\#$ is what makes this useful. To verify a stack $\ell_1; \dots; \ell_n$ against a specification at the far end, push that specification back through the passes one at a time and check each pass against whatever arrives at it; nothing has to be proved about the composite. And nothing about the compilers changes to allow this: no new composition operator, no extra data for a pass to carry.

Remark 4. The relation was not dropped and then never missed. Among the operations Budi, Galenson and Plotkin define is a conditional, and the use they put it to is patching a compiler one cannot recompile. Assume a predicate $bug: source \rightarrow \text{bool}$ describing the sources — or a superset of them — on which an optimising compiler C_{opt} produces an incorrect target, and a simple compiler C_{simple} that is always correct; then

$$\text{IF } bug \text{ THEN } C_{\text{simple}} \text{ ELSE } C_{\text{opt}} \quad (10)$$

hides the bug. But bug is α evaluated along a compiler: $bug(S)$ asserts $\alpha(S, C_{\text{opt}}(S))$, and the hypothesis that C_{simple} is always correct is exactly the condition (7) for it to be a morphism into $\mathbf{0}$. With no relation between sources and targets available, the predicate has to live on the source type alone, and a superset is the best that can then be managed, where the exact set is a comprehension. Restoring α makes the proposal a statement with an obligation attached, and Proposition 3 says what the obligation of the composite is.

5 Copying, and compile-time effects

They report a failure whose cause the construction here makes visible. Moving compile-time effects inside, so that $f: P \rightarrow T(P')$ and $g: P \times S' \rightarrow T(S)$, does not give a category: composition fails to be associative. Their fix is to keep a single monad outside.

The diagnosis is visible in their own syntax, where the source is bound in both the reduction and the generation clause: it is used twice. Duplication is free in a cartesian setting and not free otherwise, and the general statement is the following.

Proposition 5. *A commutative comonoid in $\mathcal{D}_L(\mathcal{C})$ is a comonoid in the first coordinate together with a commutative monoid $(\emptyset, \star)$ on the second, subject to*

$$\alpha(u, \emptyset) \sqsubseteq e \quad \text{and} \quad \alpha(u, x \star x') \sqsubseteq \alpha(u, x) \otimes \alpha(u, x'). \quad (11)$$

Proof. Weakening $A \rightarrow I$ has backward map $U \times 1 \rightarrow X$, that is a point $\emptyset$ of X ; contraction $A \rightarrow A \otimes A$ has backward map $U \times (X \times X) \rightarrow X$ by (4), that is a multiplication $\star$ on X . Instantiating the morphism condition (3) for these two maps gives exactly (11), and the comonoid laws for A become the monoid laws for $(\emptyset, \star)$: counit gives unitality, coassociativity

¹In fibrational language: the functor $\mathcal{D}_L(\mathcal{C}) \rightarrow \mathcal{D}_1(\mathcal{C})$ that forgets the relation is a bifibration with posetal fibres. The base is the category of §3, the fibre over (U, X) is the poset of relations $U \times X \rightarrow L$ ordered pointwise, reindexing along ℓ is $\ell^\#$, and $\ell_!$ is its left adjoint. Nothing below uses this.

gives associativity, and cocommutativity gives commutativity. Both backward maps take u as an argument, so what one obtains is strictly a U -indexed family of monoid structures on X ; we write $\oslash$ and $\star$ for the family and ask the laws to hold uniformly in u . $\square$

The variance turns the comonoid into a monoid, and the first coordinate costs nothing. So whether a source may be duplicated is a question about the *target* type and nothing else: it must absorb two targets into one. That is what fails when effects move inside, and it is what a single outer monad restores.

Remark 6 (cases). Their CASES operation combines compilers sharing a target type, and they note that categorical sums exist when the solution objects are the same, $(P_1, S) + (P_2, S) = (P_1 + P_2, S)$. That restriction is not a convenience but exactly the hypothesis under which the sum exists. For arbitrary (U_1, X_1, α_1) and (U_2, X_2, α_2) the candidate is $(U_1 + U_2, X_1 \times X_2)$, and a mediating map into a cocone is not unique, since each injection constrains only one component of the backward map: one gets a weak coproduct only. With $X_1 = X_2 = X$ that pairing never arises. The injections are (inl, π_2) and (inr, π_2) ; given a cocone $(g_i, G_i): (U_i, X, \alpha_i) \rightarrow (Q, R, \gamma)$, the mediator must be $[g_1, g_2]$ forwards and, since coproducts in a cartesian closed $\mathcal{C}$ are distributive, $[G_1, G_2]$ backwards, uniquely; and (3) holds summand by summand for $[\alpha_1, \alpha_2]$. So $(U_1 + U_2, X, [\alpha_1, \alpha_2])$ is a genuine coproduct.

Remark 7 (order). Contraction is symmetric, so Proposition 5 asks for a *commutative* monoid, and a target type that assembles its two arguments in a fixed order does not qualify. Their own sequential partial compiler is such a type: it splits a source into a prefix and a suffix and generates the composite target from the two it receives, in that order. So the pass that motivates their star operation is one whose sources cannot be duplicated, and whether the right notion here is a commutative monoid or a monoid — that is, whether the copying available to a pass should be sensitive to the order in which its children’s targets are assembled — is left open.

The construction has a second reading, in which U, X are the states and V, Y the views of a bimorphic lens, and there Proposition 5 accounts for the hierarchy of optics: which of lens, traversal and affine traversal admits copying is decided by the monoid available on the update type. That is a separate story, told elsewhere.

6 Cost

The monoid on the target type and the $\otimes$ of the lineale are not independent: an equation satisfied by the first is inherited by the second along the image of α . Suppose $\star$ is idempotent, $x \star x = x$. Then (11) gives

$$\alpha(u, x) \sqsubseteq \alpha(u, x) \otimes \alpha(u, x),$$

so every value α takes lies below its own square. If the lineale is affine — e the top, so that $a \otimes b \sqsubseteq a \wedge b$ — the reverse inequality holds as well and α takes values among the $\otimes$ -idempotents; if all of L is idempotent then $\otimes = \wedge$ and the lineale is cartesian. Over the Lawvere quantale the same computation reads $a \geq a + a$, forcing $a = 0$: an idempotent target type makes every target free. A free monoid on the target type imposes no equation at all, and leaves the lineale unconstrained.

Where the lineale is free to be quantitative, the specification becomes a cost model. The Lawvere quantale is $[0, \infty]$ with the order *reversed*, $\otimes = +$ and $e = 0$; relations valued in it are the generalised metric spaces of [4], where the reversal is what lets a L -valued relation be read as a distance and $\otimes$ as the triangle inequality. Over it, with $\alpha(S, T)$ the size — or the running time, or the register pressure — of the target T for the source S , condition (3) says that the target a pass assembles costs at least what its child's target cost — a pass may add but never remove — and Proposition 3 computes the bound for a stack from the bounds of its passes. Specifications of this shape, with equations holding up to a numerical distance rather than on the nose, are the subject of quantitative algebraic reasoning [6]; the related discipline of tracking a resource annotation in the types themselves is quantitative type theory [5]. This is the setting their multi-machine partitioning examples ask for, and it is available at no extra cost provided the target type is free.

Their star operation supplies the example that makes the point. A compiler is lifted to lists of sources by applying it pointwise, and to carry a specification along with it one needs a relation on lists — that is, a way of scoring a list of values of L . One is already to hand, since $(L, \otimes, e)$ is a monoid:

$$\alpha^*(\vec{u}, \vec{x}) = \bigotimes_i \alpha(u_i, x_i), \quad (12)$$

with e on the empty list. Over the Lawvere quantale (12) is the total cost of a list of targets. Note where the free monoid has gone: in §5 it was a structure asked of the target type, and here it is the aggregation of a list of values of L . The two are the same construction on opposite sides of the relation, and the second is what makes the sequential partial compiler of Remark 7 carry a cost model even though it cannot copy.

7 The type of verified partial compilers

Nothing on the target side can restrict the source: a specification cuts down which sources a pass is meant to handle, and that is an operation on the first coordinate. Under the reading fixed in §4, a source meets its specification when nothing refutes it, so for $\delta \in L$ adjoin

$$\mathbf{Cmp}_\delta(U, X, \alpha) = (\{u \mid \bigvee_x \alpha(u, x) \sqsubseteq \delta\}, X, \alpha), \quad (13)$$

an idempotent comonad: adjoining it is passing to the objects all of whose sources are refuted by nothing worse than δ . Taking $\delta = \perp$ gives the sources with no counterexample at all, and $\mathbf{Cmp}_\perp(U, X, \alpha)$ is the object whose witnesses are exactly those u for which every compiler into $\mathbf{0}$ out of u is correct, in the sense of (7). Taking δ arbitrary is what §6 needs: over the Lawvere quantale $\perp$ is ∞ and the condition is vacuous, whereas a general δ reads as a budget, and $\mathbf{Cmp}_\delta$ becomes a family of comonads indexed by it.

The affirmative form of this definition, $e \sqsubseteq \bigwedge_x \alpha(u, x)$, says the same thing when L is complemented and something else when it is not; §4 records why the two are interchangeable exactly to the extent that α is decidable.

Suppose now that the target type carries the error monad $X \mapsto X + 1$, so that a pass may decline to produce a target. This is not an extra assumption so much as an acknowledgement of practice: a pass that cannot handle a source has to say so.

Conjecture 8. With targets in $X + 1$ and $\mathbf{Cmp}_\delta$ available, the object $[A, B]$ of *verified* partial compilers from A to B exists, and is right adjoint to $- \times A$, for the product of (5), whose target type is $X + Y$.

The witness object of $[A, B]$ would be cut down to those pairs that satisfy their own condition, making it the type of passes that meet the specification rather than a type of candidates with a side condition attached; and the adjunction would be cartesian rather than monoidal. The option of declining is what lets a pair survive the comprehension, since without it the object collapses to the totally verified passes — which is the same structure Proposition 5 says blocks copying. A pass that may fail cannot duplicate its source.

We state this as a conjecture rather than a theorem. It is the one claim here that is not settled by unfolding a definition, and the comprehension comonad deserves an exposition this note does not give it.

Remark 9. Any such adjunction is likely to hold only up to identification of morphisms that agree wherever their conditions are not vacuous. Whether to quotient is a decision about which category is under study; it is not settled here.

8 What this is

Everything above except Conjecture 8 is elementary: each statement is checked by unfolding a definition, and none of it needs dependent types, fibrational machinery, or a reformulation of the notion of partial compiler. The compilers are the ones already written; the relation is what was thrown away when the notion was extracted, and putting it back gives correctness, a criterion for duplication, a cost model, and a type of verified passes.

The system that prompted the notion has gone: DryadLINQ is no longer maintained, and the cluster it targeted is not the machine anyone compiles for now. The shape of the problem has not gone. Splitting a program, handing the pieces to compilers that know about different targets, and reassembling what they return is the ordinary architecture for heterogeneous and distributed compilation — nested intermediate representations lowered one into the next, machine-learning graphs divided between accelerator and host backends, query plans compiled in stages. What such an architecture does not express, and what is instead maintained outside the types in test suites and comments, is the claim that a stage introduces no errors of its own. That claim is (3), and it costs one relation.

Three things are open, and all are named above: the ordering of Remark 7, for which dropping the symmetry of $\otimes$ and admitting two residuals, as in the Lambek setting, looks like the way out; the quotient of Remark 9; and Conjecture 8 itself.

References

- [1] M. Budiu, J. Galenson and G. D. Plotkin. The compiler forest. In *ESOP 2013*, LNCS 7792, pages 21–40. Springer, 2013.
- [2] V. de Paiva. The Dialectica categories. In *Categories in Computer Science and Logic*, Contemporary Mathematics 92, pages 47–62. American Mathematical Society, Providence, RI, 1989.
- [3] V. de Paiva. *The Dialectica categories*. Technical Report 213, University of Cambridge Computer Laboratory, 1991.

- [4] F. W. Lawvere. Metric spaces, generalized logic, and closed categories. *Rendiconti del Seminario Matematico e Fisico di Milano*, 43:135–166, 1973. Reprinted in *Reprints in Theory and Applications of Categories*, 1:1–37, 2002.
- [5] R. Atkey. Syntax and semantics of quantitative type theory. In *LICS 2018*. ACM, 2018.
- [6] R. Mardare, P. Panangaden and G. D. Plotkin. Quantitative algebraic reasoning. In *LICS 2016*. ACM, 2016.
- [7] V. de Paiva. Partial compilers and Dialectica spaces. Talk at BACAT, September 2011. <https://www.slideshare.net/slideshow/partial-compilers/11320612>
- [8] J. Hedges. Lenses for philosophers. Blog post, 16 August 2018. <https://julesh.com/posts/2018-08-16-lenses-for-philosophers.html>
- [9] J. Hedges. The game semantics of game theory. arXiv:1904.11287, 2019.
- [10] V. de Paiva and S. G. da Silva. Kolmogorov-Veloso problems and Dialectica categories. arXiv:2107.07854, 2021.